SIMATS ENGINEERING — Saveetha Institute of Medical and Technical Sciences

Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	192525223
Student Name	T. Srija

Assumption: servers execute 10^8 basic operations per second; nightly window = 10 min = 600 s = 6×10^{10} operations. All logarithms are base 2 unless stated. Current n = 10^5, projected n = 10^6.

(a) Understanding the Problem

Module	Input size n	Basic operation and where it occurs	Expected growth
M1 Duplicate detector	Number of products in the catalogue	The IF test comparing title and seller of Product[i] and Product[j], executed once per (i, j) pair in the inner loop. It is the most frequent statement.	One execution per pair of products → quadratic, $n^2/2$
M2 Rate limiter	Upper limit n of the request window	Audit(Request[j]) in the inner FOR loop (marked as basic operation). Inner loop runs i times, where i doubles each outer pass.	Sum of a geometric series 1+2+4+… → linear
M3 Search (S1, S2)	Number of products n in the catalogue / sorted index	Key comparison (Product.key = target in S1, key vs middle entry in S2).	S1: up to n (linear). S2: halves the range each time → logarithmic
M4 Recommendation engine	Size of the customer-product data	The linear merge step (one unit of work per element merged), f(n) = n at every recursive call; total work = sum of f(n) over the whole recursion tree.	Current: 4 subproblems of half size → quadratic. Redesign: n log n
M5 Sales analytics (and report generator)	**Number of sales records n**	**Record comparison while sorting by revenue at each level, f(n) = n log n per call. Report generator: $f(n) = 3n^2 + 10n \log n + 50$, dominated by the n^2 term.**	**$n \log^2 n$ for M5; n^2 for the report generator**

(b) Analysis of the Iterative Modules

M1 – exact number of comparisons

For a fixed i, j runs from i+1 to n−1, i.e. n−1−i iterations. The IF statement is the basic operation, so

$$C(n) = \Sigma_{i=0}^{n-2} (n-1-i) = (n-1) + (n-2) + \ldots + 1 = n(n-1)/2 = n^2/2 - n/2$$

The count is the same for best, average and worst case because there is no early exit. Time: $\Theta(n^2)$. Space: $\Theta(1)$ auxiliary (only the indices i, j; the product array is the input). For $n = 10^5$: 4,999,950,000 $\approx 5 \times 10^9$ comparisons (≈ 50 s). For $n = 10^6$: 499,999,500,000 $\approx 5 \times 10^{11}$ (≈ 83 min). A 10× growth in n gives a 100× growth in cost.

M2 – exact number of Audit operations

The outer loop takes i = 1, 2, 4, …, 2^k while i < n. The inner loop runs i times (j = 0…i−1), not n times. The values of i are $2^0, 2^1, \ldots, 2^{m-1}$, where $m = \lceil \log_2 n \rceil$ is the number of passes (the smallest m with $2^m \geq n$).

$$A(n) = \Sigma_{k=0}^{m-1} 2^k = 2^m - 1 = 2^{\lceil \log_2 n \rceil} - 1$$

Since $n \leq 2^{\lceil \log n \rceil} < 2n$, we get $n - 1 \leq A(n) \leq 2n - 2$. For n a power of two, A(n) = n − 1 exactly.

Is the claim $\Theta(n \log n)$ correct? No. The team member assumed that the inner loop runs n times in every pass. In fact it runs only i times, and i doubles, so the pass costs form a geometric series 1 + 2 + 4

+ … whose sum is dominated by its last term (about n/2 to n). Only the last pass is of size Θ(n); the earlier passes together cost no more than the last one. Hence $n - 1 \le A(n) \le 2n - 2$, giving $A(n) = \Theta(n)$. Since n log n is not O(n), the Θ(n log n) claim is wrong; it is an overestimate (it is a valid but loose upper bound, O(n log n), but not tight). Tight bound: Θ(n). Space: Θ(1).
Verification for n = 1,024 (= 2^{10}): i takes 1, 2, 4, …, 512 (i = 1024 fails i < n), i.e. 10 passes = $\log_2$ 1024. A = 1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256 + 512 = $2^{10} - 1$ = 1,023 Audit operations. The claimed n log n would give 1,024 × 10 = 10,240, which is about 10 times too large. (Check for n = 10^6: m = 20, A = $2^{20} - 1$ = 1,048,575.)

(c) Solving the Recurrence for S2 by Backward Substitution

T(n) = T(n/2) + 1, T(1) = 1. Take $n = 2^k$.

T(n) = T(n/2) + 1
= [T(n/4) + 1] + 1 = T(n/2^2) + 2
= [T(n/8) + 1] + 2 = T(n/2^3) + 3
… after i substitutions: T(n) = T(n/2^i) + i

The recursion stops when $n/2^i = 1$, i.e. $i = k = \log_2 n$:

$$T(n) = T(1) + \log_2 n = 1 + \log_2 n$$

For general n the closed form is $T(n) = \lfloor \log_2 n \rfloor + 1$. Check: T(1) = 1 + 0 = 1; T(2) = T(1) + 1 = 2 = 1 + 1; T(8) = 4 = 1 + 3; T(1,000,000) = 20. Time complexity: Θ(log n).

Implementation	Time	Space
Iterative binary search (low, high, mid variables, loop)	Θ(log n)	Θ(1)
Recursive binary search (one call per halving)	Θ(log n)	Θ(log n) – call stack depth ⌊log n⌋ + 1 (unless tail-call optimised)

(d) Master Theorem

Master Theorem for T(n) = aT(n/b) + f(n): compare f(n) with $n^{\log_b a}$. Case 1: $f(n) = O(n^{\log_b a - \epsilon}) \Rightarrow \Theta(n^{\log_b a})$. Case 2: $f(n) = \Theta(n^{\log_b a}) \Rightarrow \Theta(n^{\log_b a} \log n)$. Case 3: $f(n) = \Omega(n^{\log_b a + \epsilon})$ and regularity $af(n/b) \le cf(n)$, $c < 1 \Rightarrow \Theta(f(n))$.

	Current M4	Redesigned M4
Recurrence	T(n) = 4T(n/2) + n	T(n) = 2T(n/2) + n
a, b, f(n)	a = 4, b = 2, f(n) = n	a = 2, b = 2, f(n) = n
$n^{\log_b a}$	$n^{\log_2 4} = n^2$	$n^{\log_2 2} = n$
Comparison	$f(n) = n = O(n^{2-\epsilon})$ with ϵ = 1 (f is polynomially smaller)	f(n) = n = Θ(n) (same order)
Case	Case 1	Case 2
Result	Θ(n^2)	Θ(n log n)

Effect of doubling the data. Current design: $T(2n)/T(n) \to 2^2 = 4\times$ (exact solution $T(n) = 2n^2 - n$ gives $T(2n) \approx 4T(n)$). Redesign: T(2n) = 2n log(2n) = 2n log n + 2n, i.e. slightly more than 2× (ratio 2(1 + 1/log n), ≈ 2.1 at $n = 10^6$). Removing replication changes growth from quadratic to near-linear.

M5 – T(n) = 2T(n/2) + n log n

a = 2, b = 2, so $n^{\log_2 2} = n$ and f(n) = n log n.

- Case 1: needs $f(n) = O(n^{1-\epsilon})$ – no, f is larger than n.
- Case 2: needs f(n) = Θ(n) – no, n log n grows faster than n.
- Case 3: needs $f(n) = \Omega(n^{1+\epsilon})$ for some ϵ > 0 – no, because $(n \log n)/n^{1+\epsilon} = \log n / n^{\epsilon} \to 0$. f(n) is larger than n only by a log factor, not polynomially.

No standard case applies (f falls in the gap between Case 2 and Case 3). Use the extended Master Theorem: if $f(n) = \Theta(n^{\log_b a} \log^k n)$ with k ≥ 0, then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$. Here k = 1, so T(n) = Θ(n log² n).

Recursion-tree verification ($n = 2^L$, $L = \log n$): level i has 2^i nodes of size $n/2^i$, each costing $(n/2^i)\log(n/2^i)$. Level cost $= n(L - i)$. Levels $i = 0 \ldots L-1$ sum to

$$\Sigma\, n(L - i) = n \cdot [L + (L-1) + \ldots + 1] = n \cdot L(L+1)/2$$
Leaves: $n \cdot T(1) = n \Rightarrow T(n) = n \cdot L(L+1)/2 + n = \Theta(n \log^2 n)$

(e) Best, Average and Worst Cases

	Best case	Average case	Worst case
S1 sequential (unsorted)	Key is the first item: 1 comparison	Key present, equally likely at any position: $(n+1)/2$	Key is last item or absent: n comparisons
S2 binary (sorted index)	Key equals the first middle entry: 1 comparison	$\Theta(\log n)$; successful search $\approx \log_2 n - 1$ comparisons ($n = 10^6 \approx 19$)	Key absent or found at the last level: $\lfloor \log_2 n \rfloor + 1$ ($n = 10^6$: 20)

S1 – average-case derivation with probability p

Successful search: the key is at position i ($i = 1 \ldots n$) with probability p/n each, costing i comparisons. Unsuccessful (probability $1 - p$): all n elements are compared.

$$C_{avg}(n) = \Sigma_{i=1}^{n}\, i \cdot (p/n) + n(1 - p) = (p/n) \cdot n(n+1)/2 + n(1 - p)$$
$$= p(n+1)/2 + n(1 - p)$$

Checks: $p = 1$ gives $(n+1)/2$; $p = 0$ gives n. For $n = 1{,}000{,}000$ and $p = 0.5$:

$$C_{avg} = 0.5 \times 1{,}000{,}001/2 + 1{,}000{,}000 \times 0.5 = 250{,}000.25 + 500{,}000 = 750{,}000.25 \text{ comparisons } (\approx 0.75\, n)$$

For comparison, S2 needs at most 20 comparisons for the same catalogue, so S2 is about 37,500 times cheaper on average (provided the index is kept sorted).

(f) Proof that $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$

Definition: $f(n) = \Theta(n^2)$ if there exist constants $c_1, c_2 > 0$ and n_0 such that $c_1 n^2 \le f(n) \le c_2 n^2$ for all $n \ge n_0$.

Lower bound. For $n \ge 1$, $10n \log n \ge 0$ and $50 > 0$, so $f(n) \ge 3n^2$. Take $c_1 = 3$.

Upper bound. For $n \ge 1$: $\log_2 n \le n$, so $10n \log n \le 10n^2$; and $50 \le 50n^2$. Hence $f(n) \le 3n^2 + 10n^2 + 50n^2 = 63n^2$. Take $c_2 = 63$.

Therefore $3n^2 \le f(n) \le 63n^2$ for all $n \ge n_0 = 1$, so $f(n) = \Theta(n^2)$. (Check: $n = 1$: $f = 53$, bounds $3 \le 53 \le 63$. $n = 10$: $f \approx 300 + 332 + 50 = 682$, bounds $300 \le 682 \le 6{,}300$.) ■

$f(n)$ is not $O(n)$. Suppose, for contradiction, $f(n) \le c \cdot n$ for some constants $c > 0$ and all $n \ge n_0$. Since $f(n) \ge 3n^2$, we get $3n^2 \le cn$, i.e. $n \le c/3$ for every $n \ge n_0$. But choosing $n = \max(n_0, \lceil c/3 \rceil + 1)$ violates this. Contradiction, so $f(n) \notin O(n)$. (Equivalently, $f(n)/n \ge 3n \to \infty$.) ■

(g) Scalability Decision for n = 1,000,000

Using exact or leading-term counts ($\log_2 10^6 \approx 19.93$). Time = operations ÷ 10^8. Window limit: 600 s (6×10^{10} operations).

Module	Operations at $n = 10^6$	Time	Within 10 min?
M1 duplicate detector, $n(n-1)/2$	$\approx 5.0 \times 10^{11}$	$\approx$ 5,000 s $\approx$ 83.3 min	No (8× over)
M2 rate limiter, $2^{20} - 1$	1,048,575 $\approx 1.05 \times 10^6$	$\approx$ 0.0105 s	Yes
S2 binary search (per query)	20 comparisons	$\approx 2 \times 10^{-7}$ s	Yes
M4 current, $T = 2n^2 - n$	$\approx 2.0 \times 10^{12}$	$\approx$ 20,000 s $\approx$ 5.6 h	No (33× over)
M4 redesign, $T = n \log n + n$	$\approx 2.1 \times 10^7$	$\approx$ 0.21 s	Yes
M5, $n \cdot L(L+1)/2 + n$	$\approx 2.10 \times 10^8$	$\approx$ 2.1 s	Yes
Report generator, $3n^2 + 10n \log n + 50$	$\approx 3.0 \times 10^{12}$	$\approx$ 30,000 s $\approx$ 8.3 h	No (50× over)

(Solutions of M4 use $T(1) = 1$: $T(n) = 2n^2 - n$ for the current design and $T(n) = n \log_2 n + n$ for the redesign; the closed forms are exact for n a power of 2 and a close estimate for $n = 10^6$.) Growth from $n = 10^5$ to 10^6: M1 and the current M4 and report generator grow $\approx 100\times$; M2 $\approx 10\times$; M4 redesign and M5 $\approx$ 11–13×; S2 grows only from 17 to 20 comparisons.

Recommendations

- M1 – must be redesigned. Replace pairwise comparison by hashing: insert the key (title, seller) in a hash table and flag a product when the key already exists. Expected time $\Theta(n)$ ($\approx 10^6$ operations, $\approx$ 0.01 s), space $\Theta(n)$. Alternative: sort by (title, seller) in $\Theta(n \log n)$ and compare neighbours, with no hash table needed.
- M4 – must be redesigned. Adopt the proposed non-replicated design, $T(n) = 2T(n/2) + n = \Theta(n \log n)$: about 10^5 times fewer operations ($2 \times 10^{12} \rightarrow 2 \times 10^7$), and doubling the data costs only $\approx 2\times$ instead of $4\times$.
- Report generator – must be redesigned (not required by the question, but its n^2 term makes it fail the window at $\approx$ 8.3 h). Profile the quadratic loop and replace it with a sort/hash/aggregation approach, targeting $O(n \log n)$.
- M2, S2, M5 – may stay. M2 is linear ($\approx$ 0.01 s), S2 is logarithmic (20 comparisons), and M5 takes $\approx$ 2 s. Optional improvement for M5: sort once (or merge already-sorted halves) to reach $\Theta(n \log n)$ instead of $\Theta(n \log^2 n)$.
- M3 – use S2. S1 averages $\approx$ 750,000 comparisons per query at $n = 10^6$ and degrades linearly, whereas S2 needs at most 20. Keep the index sorted (or use a balanced tree / hash index if frequent inserts are expected).

Code of Conduct

I T. Srija / 192525223 (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: T. Srija Date: 13-10-2026